

Анализ графов

Пентегова Мария Сергеевна

Группа: 25.Б41-мм

Кафедра: системного программирования

Научный руководитель: Григорьев Семён Вячеславович

Номер семестра практики: 2

1. Введение

GraphBLAS позволяет формулировать алгоритмы анализа графов в терминах операций линейной алгебры над разреженными матрицами и векторами. В данной работе реализован обход графа в ширину (BFS) в классическом и матрично-алгебраическом вариантах. Цель эксперимента — исследовать влияние структуры графа на производительность этих реализаций.

2. Постановка задачи и реализация

Реализованы четыре варианта BFS: классический BFS с построением массива родителей (parent-BFS), классический multisource BFS, GraphBLAS BFS с вычислением уровней и GraphBLAS multisource BFS. Для GraphBLAS используется SuiteSparse:GraphBLAS; классическая реализация необходима как контрольный вариант для сравнения и проверки корректности результатов.

В классическом варианте граф хранится в формате CSR (Compressed Sparse Row). Для вершины v её соседи находятся в диапазоне

$$[\text{row_ptr}[v], \text{row_ptr}[v + 1])$$

массива `col_idx`. BFS использует очередь, а при первом посещении вершины u записывается `parent[u] = v`. Для multisource BFS очередь изначально содержит все исходные вершины. Сложность классического BFS составляет $O(|V| + |E|)$.

В GraphBLAS граф представляется булевой матрицей смежности A , а текущий фронт — разреженным булевым вектором f_k . Следующий фронт вычисляется как

$$f_{k+1} = f_k A$$

с использованием булевого semiring LOR-LAND. Уже посещённые вершины исключаются маской `visited`. Номер итерации определяет уровень BFS. В multisource BFS начальный вектор содержит несколько источников, после чего уровни вычисляются той же последовательностью матрично-векторных операций.

Такой подход непосредственно соответствует назначению GraphBLAS: графовые алгоритмы представляются через операции над матрицами и векторами с использованием обобщённых алгебраических структур [1].

Программа написана на C11. Для сборки используется CMake, для тестирования — CTest, для измерения времени — `POSIX clock_gettime()` с `CLOCK_MONOTONIC`. Матрица GraphBLAS импортируется из подготовленного CSR-представления. Эксперименты выполнены с SuiteSparse 7.14.1 и SuiteSparse:GraphBLAS 10.3.2 [3, 2].

3. Эксперимент

Для экспериментов выбраны семь графов из SuiteSparse Matrix Collection, представляющей набор разреженных матриц из различных прикладных областей [4]. Для однократного BFS источником выбиралась вершина с максимальной степенью. Для multisource BFS использовались четыре источника, равномерно распределённых по множеству вершин. Перед измерениями выполнялся прогревочный запуск, после чего каждая реализация запускалась 10 раз.

Граф	$ V $	$ E $	Тип	$\deg(s)$
com-Orkut	3.07M	117M	социальный	33313
soc-LiveJournal1	4.85M	69M	социальный	38660
road_usa	23.95M	28.85M	дорожный	4
roadNet-CA	1.97M	2.77M	дорожный	3
web-Google	0.92M	5.10M	веб-граф	6332
cit-Patents	3.77M	16.52M	цитирование	852
com-Amazon	0.33M	0.93M	социальный	546

Таблица 1: Характеристики использованных графов.

Время измерялось в микросекундах. В таблице приведены средние значения по 10 запускам.

Граф	CP	CM	GB-L	GB-M	CP/GB-L
com-Orkut	2652628	2700768	486500	420307	5.45x
soc-LiveJournal1	1915627	1976468	358255	331458	5.35x
road_usa	1451387	1588466	25839879	22121848	0.06x
roadNet-CA	127543	110558	663630	670218	0.19x
web-Google	100233	80683	101149	106972	0.99x
cit-Patents	17500	4730	181639	46757	0.10x
com-Amazon	38967	31974	61923	62144	0.63x

Таблица 2: Среднее время BFS по 10 запускам, мкс. CP — Classic Parent, CM — Classic Multisource, GB-L — GraphBLAS Level, GB-M — GraphBLAS Multisource.

4. Анализ результатов

На `com-Orkut` и `soc-LiveJournal1` GraphBLAS Level показал меньшее время: 486 500 и 358 255 мкс против 2 652 628 и 1 915 627 мкс для CP. Отношение времени CP к GB-L составляет 5.45 и 5.35 соответственно.

На `road_usa` и `roadNet-CA`, напротив, время GB-L составило 25 839 879 и 663 630 мкс против 1 451 387 и 127 543 мкс для CP. На `web-Google` времена близки: 100 233 и 101 149 мкс. На `cit-Patents` CP также быстрее: 17 500 против 181 639 мкс. Для `com-Amazon` получено 38 967 против 61 923 мкс.

Таким образом, в исследованной выборке преимущество реализаций зависит от конкретного графа. Одним из возможных факторов является структура BFS-фронт: широкие фронты позволяют выполнять больше работы одной матричной операцией, тогда как при узких фронтах накладные расходы GraphBLAS могут быть существеннее. В данной работе ширина фронтов не измерялась, поэтому это объяснение является гипотезой для дальнейшего эксперимента. Ориентированность графа также не является достаточным объяснением: например, `cit-Patents` и социальные графы из выборки демонстрируют различное соотношение времён.

5. Тестирование

Корректность проверяется с помощью `CTest` и `assert.h`. Тесты выполняются на небольшом графе и проверяют посещение вершин, вычисление расстояний, построение родительского дерева, multisource BFS и соответствие результатов классической и GraphBLAS-реализаций. Все тесты проходят успешно: **100% passed**. `CTest` предоставляет механизм запуска и проверки тестов, определённых в проекте `CMake` [5].

6. Заключение

В работе реализованы четыре варианта BFS на C: два классических варианта на CSR и два варианта на SuiteSparse:GraphBLAS. Классический BFS использован как контролируемый baseline, позволяющий сравнивать два способа представления одного алгоритма и проверять корректность GraphBLAS-реализации.

Эксперименты на семи графах из SuiteSparse Matrix Collection показали, что время выполнения зависит от структуры графа. На `com-Orkut` и `soc-LiveJournal1` GraphBLAS Level был быстрее CP примерно в 5.45 и 5.35 раза, тогда как на `road_usa`, `roadNet-CA`, `cit-Patents` и `com-Amazon` меньшее время показал CP. На `web-Google` результаты были близки. Поэтому по результатам данной выборки нельзя вывести универсальное преимущество одного подхода.

Для продолжения эксперимента следует измерять размеры BFS-фронт на каждом уровне, число обработанных рёбер и накладные расходы операций GraphBLAS. Это позволит проверить связь структуры фронтов с наблюдаемой разницей производительности.

Репозиторий: https://github.com/MariaPentegova/graph_blas_bfs-2.

Список литературы

- [1] GraphBLAS Forum. *The GraphBLAS C API Specification. Version 2.1*, 2023. https://graphblas.org/docs/GraphBLAS_API_C_v2.1.0.pdf
- [2] Davis, T. A. *SuiteSparse:GraphBLAS*, release 10.3.2, 2026. <https://github.com/DrTimothyAldenDavis/GraphBLAS/releases>
- [3] Davis, T. A. *SuiteSparse*, version 7.14.1, 2026. <https://github.com/DrTimothyAldenDavis/SuiteSparse>
- [4] Davis, T. A., Hu, Y. *The University of Florida Sparse Matrix Collection*. ACM Transactions on Mathematical Software, 38(1), 2011. <https://sparse.tamu.edu/>
- [5] Kitware. *CTest Documentation*, CMake Documentation, 2026. <https://cmake.org/cmake/help/latest/module/CTest.html>
- [6] The Open Group. *The Open Group Base Specifications Issue 8. POSIX.1*, 2024. <https://pubs.opengroup.org/onlinepubs/9799919799/>